

Fold#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.fold.Fold.html>

Description:

Combines an array of sliding local blocks into a large containing tensor. Consider a batched input tensor containing sliding local blocks, e.g., patches of images, of shape $(N, C \times \prod(\text{kernel_size}), L)$ $(N, C \times \prod(\text{kernel_size}), L)$, where N is batch dimension, $C \times \prod(\text{kernel_size})$ is the number of values within a block (a block has $\prod(\text{kernel_size})$ spatial locations each containing a C -channeled vector), and L is the total number of blocks. (This is exactly the same specification as the output shape of `Unfold`.) This operation combines these local blocks into the large output tensor of shape $(N, C, \text{output_size}[0], \text{output_size}[1], \dots)$ by summing the overlapping values. Similar to `Unfold`, the arguments must satisfy where ddd is over all spatial dimensions. `output_size` describes the spatial shape of the large containing tensor of the sliding local blocks. It is useful to resolve the ambiguity when multiple input shapes map to same number of sliding blocks, e.g., with `stri`

Function Signature

```
class torch.nn.modules.fold.Fold(output_size, kernel_size,  
dilation=1, padding=0, stride=1)[source]#
```

Parameters

Shape:

```
extra_repr()[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> fold_params = dict(kernel_size=..., dilation=...,  
padding=..., stride=...)  
>>> fold = nn.Fold(output_size=..., **fold_params)  
>>> unfold = nn.Unfold(**fold_params)
```

```
fold(unfold(input)) == divisor * input
```

```
>>> input_ones = torch.ones(input.shape, dtype=input.dtype)  
>>> divisor = fold(unfold(input_ones))
```

```
>>> fold = nn.Fold(output_size=(4, 5), kernel_size=(2, 2))  
>>> input = torch.randn(1, 3 * 2 * 2, 12)  
>>> output = fold(input)  
>>> output.size()  
torch.Size([1, 3, 4, 5])
```

Notes & Warnings

NOTE

Note Fold calculates each combined value in the resulting large tensor by summing all values from all containing blocks. Unfold extracts the values in the local blocks by copying from the large tensor. So, if the blocks overlap, they are not inverses of each other. In general, folding and unfolding operations are related as follows. Consider Fold and Unfold instances created with the same parameters: `>>> fold_params = dict(kernel_size=..., dilation=..., padding=..., stride=...) >>> fold = nn.Fold(output_size=..., **fold_params) >>> unfold = nn.Unfold(**fold_params)` Then for any (supported) input tensor the following equality holds: `fold(unfold(input)) == divisor * input` where divisor is a tensor that depends only on the shape and dtype of the input: `>>> input_ones = torch.ones(input.sh`

⚠ WARNING

Warning Currently, only unbatched (3D) or batched (4D) image-like output tensors are supported.

Detailed Documentation

Documentation

Combines an array of sliding local blocks into a large containing tensor.

Consider a batched input tensor containing sliding local blocks, e.g., patches of images, of shape $(N, C \times \prod(\text{kernel_size}), L)$ $(N, C \times \prod(\text{kernel_size}), L)$

$(N, C \times \prod(\text{kernel_size}), L)$, where NNN is batch dimension, $C \times \prod(\text{kernel_size})$ is the number of values within a block (a block has $\prod(\text{kernel_size})$ spatial locations each containing a C -channeled vector), and LLL is the total number of blocks. (This is exactly the same specification as the output shape of `Unfold`.) This operation combines these local blocks into the large output tensor of shape $(N, C, \text{output_size}[0], \text{output_size}[1], \dots)$ by summing the overlapping values. Similar to `Unfold`, the arguments must satisfy

where ddd is over all spatial dimensions.

`output_size` describes the spatial shape of the large containing tensor of the sliding local blocks. It is useful to resolve the ambiguity when multiple input shapes map to same number of sliding blocks, e.g., with `stride > 0`.

The `padding`, `stride` and `dilation` arguments specify how the sliding blocks are retrieved.

`stride` controls the stride for the sliding blocks.

`padding` controls the amount of implicit zero-paddings on both sides for padding number of points for each dimension before reshaping.

`dilation` controls the spacing between the kernel points; also known as the *à trous* algorithm. It is harder to describe, but this link has a nice visualization of what dilation does.

`output_size` (int or tuple) – the shape of the spatial dimensions of the output (i.e., `output.size()[2:]`)

`kernel_size` (int or tuple) – the size of the sliding blocks

dilation (int or tuple, optional) – a parameter that controls the stride of elements within the neighborhood. Default: 1

padding (int or tuple, optional) – implicit zero padding to be added on both sides of input. Default: 0

stride (int or tuple) – the stride of the sliding blocks in the input spatial dimensions. Default: 1

If output_size, kernel_size, dilation, padding or stride is an int or a tuple of length 1 then their values will be replicated across all spatial dimensions.

For the case of two output spatial dimensions this operation is sometimes called col2im.

Fold calculates each combined value in the resulting large tensor by summing all values from all containing blocks. Unfold extracts the values in the local blocks by copying from the large tensor. So, if the blocks overlap, they are not inverses of each other.

In general, folding and unfolding operations are related as follows. Consider Fold and Unfold instances created with the same parameters:

Then for any (supported) input tensor the following equality holds:

where divisor is a tensor that depends only on the shape and dtype of the input:

When the divisor tensor contains no zero elements, then fold and unfold operations are inverses of each other (up to constant divisor).

Currently, only unbatched (3D) or batched (4D) image-like output tensors are supported.

Input: $(N, C \times \prod (\text{kernel_size}), L)$ $(N, C \times \prod (\text{kernel_size}), L)$

$(N, C \times \prod(\text{kernel_size}), L)$ or $(C \times \prod(\text{kernel_size}), L)(C \times \prod(\text{kernel_size}), L)$

Output: $(N, C, \text{output_size}[0], \text{output_size}[1], \dots)$ or $(C, \text{output_size}[0], \text{output_size}[1], \dots)$ as described above

Return the extra representation of the module.

Runs the forward pass.